

**UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE CIÊNCIAS MATEMÁTICAS E DE COMPUTAÇÃO**

Guilherme Shinji Nakamine

**Classificação de diagnósticos pela CID-10 via textos
clínicos utilizando Processamento de Linguagem Natural
e Aprendizado de Máquina.**

São Carlos

2022

Guilherme Shinji Nakamine

**Classificação de diagnósticos pela CID-10 via textos
clínicos utilizando Processamento de Linguagem Natural
e Aprendizado de Máquina.**

Trabalho de conclusão de curso apresentado
ao Centro de Ciências Matemáticas Aplicadas
à Indústria do Instituto de Ciências Matemá-
ticas e de Computação, Universidade de São
Paulo, como parte dos requisitos para conclu-
são do MBA em Ciências de Dados.

Área de concentração: Ciências de Dados

Orientador: Profa. Dra. Gleici da Silva Castro
Perdoná

Versão original

**São Carlos
2022**

AGRADECIMENTOS

Primeiramente gostaria de agradecer aos meus pais Fukushi e Sueli e ao meu irmão Arthur que me apoiaram em todo o curso durante mais um ano difícil para o mundo inteiro devido a pandemia do corona vírus. Sem esquecer também de todos os membros da família Nakamine e Hara que me deram suporte em diversos momentos também.

Meus agradecimentos também à professora Gleici da Silva Castro Perdoná que me orientou com muita atenção e dedicação desde o início. Sempre foi muito solícita, sanou todas as dúvidas que tive e sempre indicou qual o melhor caminho a ser seguido.

Por fim, sou muito grato à Deus-Parens por me proporcionar mais um ano de saúde em que pude me dedicar aos estudos para evoluir e me tornar uma pessoa e um profissional melhor.

RESUMO

Nakamine, G. S. **Classificação de diagnósticos pela CID-10 via textos clínicos utilizando Processamento de Linguagem Natural e Aprendizado de Máquina..** 2022. 56p. Monografia (MBA em Ciências de Dados) - Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2022.

A automatização da classificação de enfermidades seguindo a Classificação Internacional de Doenças é um assunto que vem sendo amplamente estudado há alguns anos e vem ganhando cada vez mais força nos últimos anos com os novas metodologias de processamento de linguagem natural e aprendizado de máquina. Essa classificação é o padrão oficial utilizado no Brasil, porém existe pouca literatura que utiliza registros na língua portuguesa brasileira para esse fim indicando que existe um grande potencial de pesquisa nessa área. Essa é uma tarefa bastante desafiadora e complexa, já que existem centenas de milhares de registros entre categorias, conceitos e termos e mais de 1,5 milhões de termos clínicos. Além disso, são pouquíssimos profissionais que são treinados para utilizá-la. Neste contexto, este trabalho de conclusão de curso propõe o desenvolvimento de um *pipeline* completo de ciência de dados utilizando técnicas de processamento de linguagem natural e aprendizado de máquina para realizar a classificação de textos médicos segundo a CID. Esta tarefa será baseada na *Feature engineering* TF-IDF e nos modelos de regressão logística, como *base line*, e SVM. O resultado final será um *pipeline* que contém todos os passos desde a extração dos dados até a interpretação do modelo que classificar da melhor maneira o maior número possível de textos com o CID correspondente. A metodologia foi desenvolvida utilizando a linguagem Python e as bibliotecas Scikit-learn, Pandas, NLTK e Plotly. O *dataset* escolhido foi o MIMIC III em que serão utilizados apenas os códigos CID de entrada dos resumos de altas médicas e as dez categoria mais frequentes segundo o CID-10, padrão atual.

Palavras-chave: Ciência de Dados. Processamento de Linguagem Natural. Aprendizado de Máquina. Regressão Logística. Máquina de Vetores de Suporte. Classificação Internacional de Doenças. Textos clínicos.

ABSTRACT

Nakamine, G. S. **Patient diagnosis according to ICD-10 classification of diseases from clinical records using Natural Language Processing (NLP) and Machine Learning techniques..** 2022. 56p. Monografia (MBA em Ciências de Dados) - Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2022.

The automation of disease classification following the International Classification of Diseases is a subject that has been widely studied for some years and has been gaining strength in recent years with the arise of new methodologies of processing natural language and machine learning. This classification is the official standard used in Brazil, but there is little literature that uses records in Brazilian Portuguese for this purpose, indicating a great potential for research in this area. This is quite a challenging and complex task, since there are hundreds of thousands of records across categories, concepts and terms and over 1.5 million clinical terms. In addition, there are very few professionals who are trained to use it. In this context, this course conclusion paper proposes the development of a complete pipeline of data science using natural language processing and machine learning techniques to perform the classification of medical texts according to ICD standard. This task will be based on feature engineering TF-IDF and logistic regression models, as base line, and SVM. The result will be a pipeline that contains all the steps from extraction of the data until the interpretation of the model which best classify the highest possible number of texts with the corresponding CID. The methodology was developed using Python language and the Scikit-learn, Pandas, NLTK and Plotly libraries. The dataset chosen was the MIMIC III in which will be considered only the admission entry CID of medical discharge summaries and the ten most frequent categories according to the ICD-10, current official standard.

Keywords: Data Science. Natural Language Processing. Machine Learning. Logistic Regression. Support Vector Machine. Internacional Classification of Diseases. Clinical records.

LISTA DE FIGURAS

Figura 1 – Estrutura do código CID	25
Figura 2 – Representação espacial <i>SVM</i>	30
Figura 3 – Calculo da margem máxima	31
Figura 4 – <i>Schema</i> e conexões das tabelas utilizadas no <i>BigQuery</i>	39
Figura 5 – Pareto da quantidade dos 10 códigos mais frequentes.	40

LISTA DE TABELAS

Tabela 1 – Resultados de (CAO, L. et al., 2019)	20
Tabela 2 – CID-10	26
Tabela 3 – Representação binária	27
Tabela 4 – Representação pela frequência	27
Tabela 5 – Funções Kernel	32
Tabela 6 – Matriz de Confusão	33
Tabela 7 – Resultados dos modelos	41
Tabela 8 – Resultados dos modelos com Lemmatização e <i>Stemming</i>	41
Tabela 9 – Opções do <i>Grid Search</i>	42
Tabela 10 – Melhor Modelo com <i>Grid Search</i>	42
Tabela 11 – Dez palavras mais relevantes para classificação de cada CID	43

LISTA DE ABREVIATURAS E SIGLAS

NLP	<i>Natural Language Processing</i>
PLN	Processamento de Linguagem Natural
CID	Classificação Estatística Internacional de Doenças e Problemas Relacionados com a Saúde
OMS	Organização Mundial da Saúde
MIMIC	<i>Medical Information Mart for Intensive Care</i>
CAML	<i>Convolutional Attention for Multi-Label</i>
CNN	<i>Convolutional Neural Network</i>
GRU	<i>Gated Recurrent Unit</i>
ReLU	<i>Rectified Linear Unit</i>
LSTM	<i>Long Short-Term Memory</i>
HCAML	<i>Hierarchical Convolutional Attention for Multi-Label classification</i>
BiLSTM	<i>Bidirecional Long Short-Term Memory</i>
HA-GRU	<i>Hierarchical Attention-Gated Recurrent Unit</i>
RNN	<i>Recurrent Neural Network</i>
LR	<i>Logistic Regression</i>
CNN-Att	<i>Convolutional Neural Network with Attention</i>
NLTK	<i>Natural Language Toolkit</i>
SVM	<i>Support Vector Machine</i>
BOW	<i>Bag-Of-Words</i>
TF-IDF	<i>Term Frequency-Inverse Document Frequency</i>
GloVe	<i>Global Vectors</i>
AUC	<i>Area Under the ROC Curve</i>
FCBF	<i>Fast Correlation Based Filter</i>

EHR *Electronic Health Record*

SICO Sistema de Informação de Certificados de Óbito

SUMÁRIO

1	INTRODUÇÃO	17
1.1	Revisão Bibliográfica	18
1.2	Objetivo e motivação	22
2	FUNDAMENTOS TEÓRICOS	25
2.1	Código CID	25
2.2	Principais técnicas de préprocessamento de textos	26
2.3	<i>Bag of Words</i>	27
2.4	<i>Term Frequency - Inverse Document Frequency</i>	28
2.5	Regressão Logística	28
2.5.1	<i>Regressão Logística Multinomial</i>	29
2.6	<i>Support Vector Machine</i>	29
2.7	Técnicas de classificação para problemas multi-classe	32
2.7.1	<i>One-vs-Rest</i>	32
2.8	Métricas de avaliação de modelos	33
2.8.1	Acurácia	34
2.8.2	Precisão	34
2.8.3	Revocação	34
2.8.4	<i>F1-Score</i>	34
2.8.5	<i>Grid Search</i>	34
3	METODOLOGIA	35
3.1	Extração dos dados	35
3.2	Préprocessamento dos dados	36
3.3	<i>Feature engineering</i> do textos	36
3.4	Construção dos modelos	36
3.5	Avaliação dos modelos	37
3.6	Avaliação dos resultados	37
4	RESULTADOS DO TRABALHO	39
4.1	Extração da base de dados	39
4.2	Análise de Dados	40
4.3	Avaliação dos Resultados	41
4.3.1	Modelos	41
4.3.2	Lematização e <i>Stemming</i> no melhor modelo	41
4.3.3	<i>Grid Search</i>	42

4.3.4	Interpretação dos resultados	43
5	CONCLUSÕES	45
6	APÊNDICE	47
	REFERÊNCIAS	55

1 INTRODUÇÃO

A Classificação Estatística Internacional de Doenças e Problemas Relacionados com a Saúde (CID) define as doenças, sinais e sintomas, aspectos anormais e outras condições com o objetivo de padronizar e catalogar enfermidades e patologias relacionados à saúde. Ela é adotada pela Organização Mundial da Saúde (OMS) e é o padrão mais utilizado no Brasil.

Para se ter uma ideia da complexidade da classificação, de acordo com a OMS, existem aproximadamente 17.000 categorias, 80.000 conceitos, 120.000 termos e mais de 1,6 milhões de termos clínicos interpretados na CID-11 que entrará em vigor no ano de 2022. Além disso, “no caso da CID por exemplo, são muito poucos os profissionais treinados para utilizá-la.” (MORO, C. M.; JR, L. A. M. C., 2004).

A CID-10, décima edição que é a versão em vigência, é organizada por meio de quatro hierarquias: Capítulos, Agrupamentos, Categorias e Subcategorias, em que esta possui o maior nível de detalhamento.

Estudos como (MORO, C. M.; JR, L. A. M. C., 2004) que buscam medir a distância entre dois diagnósticos médicos utilizando a distância euclidiana entre códigos CID e projetos recentes como (XU, K. et al. , 2019) e (WANG, S.-m. et al., 2020) que utilizam *machine learning* para automatizar códigos CID, utilizando textos indicam avanços nessa área. Dentro desse contexto, esta pesquisa tem o objetivo de avaliar textos da área da saúde aplicados a classificação de acordo CID. Entender textos de profissionais da saúde utilizando técnicas de NLP já são e serão ainda mais fundamentais em diversos estudos que envolvam a adoção de novas tecnologias no setor da saúde.

Estudos como o desse projeto, podem avançar e possibilitar a criação de uma aplicação semelhante ao que é encontrado no software Íris, sistema de codificação de óbitos muito utilizado em países da Europa. Segundo (Pavillon, 2012) “Atualmente o Iris é utilizado como programa padrão para a codificação e seleção da causa básica na Alemanha, Austrália, Canadá, França, Noruega, Reino Unido, Suécia, Israel, República Tcheca e Luxemburgo, além de também ser usado na Catalunha. Vários países estão em processo de adoção do programa, entre eles pode-se citar a Eslovênia, Hungria e África do Sul.” (apud MARTINS, R. C., 2017).

É possível encontrar na literatura diversos trabalhos onde são apresentados alguns modelos e técnicas para classificação de acordo com o CID utilizando técnicas de *machine learning* que serão apresentados na seção 1.1.

1.1 Revisão Bibliográfica

Nessa seção serão citados algumas pesquisas já realizadas que serviram como base para realização desse trabalho de conclusão de curso sobre o tema: modelos preditivos ou de classificação utilizando textos clínicos.

Foram escolhidos pesquisas utilizando registros em diversas línguas para observar a existência de algum tipo de variação, principalmente nos procedimentos de NLP, uma vez que cada idioma possui suas particularidades. Serão apresentados nessa seção pesquisas em quatro idiomas diferentes: inglês, holandês, chinês e português.

A literatura especializada relacionada a esse tema evidencia que já foram aplicados diversos tipos modelos para prever códigos médicos de classificação como o CID e mortes. Como os objetivos de cada pesquisa são diferentes observa-se uma variedade principalmente nas premissas adotadas, na quantidade de modelos e técnicas de *NLP* utilizada em cada pesquisa e nas métricas utilizadas.

Na pesquisa de (MULLENBACH, J. et al., 2018) é apresentado uma *attentional convolutional network* que prediz os códigos baseado em registros clínicos. Os *dataset* utilizados foram duas versões do MIMIC, MIMIC-II e MIMIC-III, e a codificação utilizada foi a CID versão nove. Foi dado o nome de *Convolutional Attention for Multi-Label (CAML)* ao método que atingiu um *precision@8* de 0,71 e *Micro-F1* de 0,54 que são métricas que superaram estudos anteriores em ambos *datasets*. Além dos resultados serem melhores que o estado da arte anterior, houve uma preocupação com a interpretabilidade, uma vez que a aplicação foi considerada para se tornar um suporte de decisão. Outro fator que ajudou a melhorar a *performance* do modelo foi a utilização dos textos de descrição de cada código CID de acordo como descreve a OMS.

Em (Xu, K. et al., 2019) utilizou-se os dados do MIMIC-III e foi aplicado a técnica de *machine learning* multimodal. Em relação ao *dataset*, foram utilizados os 32 códigos CID que mais apareceram e foi feita uma conversão para a CID-10, uma vez que esses registros estão no padrão CID-9. Além disso, foi utilizado três tipos de dados textuais presentes: estruturado, semi-estruturado e não-estruturado. Para os textos não-estruturados utilizou-se o *Text-CNN* para classificação *multi-label*, para os textos semi-estruturado foi construído um modelo *deep learning* para analisar as similaridades semânticas entre o texto escrito pelo médico e a descrição dos códigos CID e, por fim, os dados estruturados foram modelados em uma *decision tree*. As duas contribuições mais relevantes dessa pesquisa foram: a junção de várias técnicas de estudos pré-existentes, formando um modelo multimodal, junto com a adoção de métodos para melhorar a interpretabilidade do modelo e dos resultados.

O foco principal da pesquisa de (WANG, S.-m. et al., 2019) é aplicar NLP e redes neurais para entender o significado por trás dos dados não-estruturados escritos

pelos médicos. O *dataset* foi obtido do Hospital Nacional da Universidade de Taiwan e é bem completo, contendo informações como histórico do tratamento, relatórios de patologia, exames e diagnósticos médicos. Os registros médicos estão escritos tanto em chinês quanto em inglês, porém como a quantidade deste é pequena, a parte na língua chinesa foi desconsiderada. Primeiramente foi realizado um pré-processamento utilizando a biblioteca *NLTK* e em seguida foi criado o *word embedding* baseado em *word2vec* e *GloVe* pela possibilidade de capturar a semântica e sintaxe transformando-as em vetores. Posteriormente esses vetores foram utilizados no primeiro *layer* da rede neural. Foi escolhido como segunda camada a GRU, pelo fato dela consumir menos tempo de processamento do que uma LSTM, e, por fim, as duas camadas remanescentes com *ReLU* e uma sigmoide como função de ativação. Como resultado obteve-se um *F1-Score* de 0,62 para a classificação do código CID-10 completo. Além disso, observou-se a alta correlação entre o tamanho dos dados e a performance, uma vez que variou-se a quantidade de dados de entrada.

No estudo de (CAO, L. et al., 2019) foi apresentado um modelo utilizando a técnica *Hierarchical Convolutional Attention for Multi-Label classification (HCAML)* de 3 camadas que prediz códigos CID de registros médicos automaticamente. Essa pesquisa mostra-se importante já que a China possui uma história recente em relação à classificação de doenças se comparado a países desenvolvidos como os EUA, algumas províncias passaram a adotar a classificação apenas em anos recentes. Por conta desse fato, atualmente existem poucos médicos especializados em codificação seguindo as normas da OMS. Além disso, existe um outro problema no país, províncias diferentes utilizam diferentes padrões CID. Apesar de serem similares, as pequenas diferenças fazem com que não haja comunicação com todo o país. O *dataset* utilizado no artigo está no idioma chinês e possui dados semi-estruturados com classificação CID-10 em todos os níveis (três, quatro e seis dígitos). Os textos em chinês são diferentes do inglês, pois não possui delimitadores entre as palavras. Dessa forma, para o pré-processamento dos registros utilizou-se a biblioteca do Python chamada *jieba* que é uma das bibliotecas de segmentação de palavras mais maduras na língua chinesa. Para melhorar a performance, não só as *stop words* e os caracteres foram removidos, como também também foi utilizado um dicionário contendo aproximadamente 100.000 termos médicos. Também foi avaliado se ajudaria a performance caso fosse adicionado *features* estruturadas como idade, gênero e departamento de admissão. O modelo *HCAML* utilizado foi baseado no *CAML* apresentada por (Mullenbach et al., 2018). A diferença é que o *HCAML* possui três *CAML* que são responsáveis por predizer os códigos 3-dígitos, 4-dígitos e 6-dígitos, respectivamente. Esses três módulos compartilham a mesma *convolution layer*, mas cada módulo possui uma *attention layer* e uma *output layer* independentes. A saída final é a concatenação de cada do resultado de cada módulo. Assim, pode-se definir esse modelo como sendo hierárquico que utiliza da hierarquia da classificação CID para melhorar a performance. A avaliação da performance do modelo foi realizada utilizando a predição do código de 6-dígitos. As métricas utilizadas foram a Hit@1 (a predição com maior

probabilidade é a classificação verdadeira), a Hit@5 (a classificação verdadeira são aquelas com as cinco maiores probabilidades) e a AUC. A primeira proposta seria para atribuir um código automaticamente e a segunda para auxiliar os profissionais reduzindo o intervalo de pesquisa. Após vários experimentos, comprovou-se que as *features* adicionais melhoram significativamente a performance e o *HCAML* funciona melhor que o modelo *baseline*, *CAML*, em todos os cenários. Os resultados obtidos superaram o modelo "estado-da-arte" e estão representados na tabela 1.

Tabela 1 – Resultados de (CAO, L. et al., 2019)

Model	Hit@1	Hit@5	AUC
CAML	0,6611	0,8432	0,9444
CAML + <i>Features</i>	0,687	0,8818	0,9376
HCAML	0,6974	0,8918	0,9514
HCAML + <i>Features</i>	0,6969	0,8903	0,9496

Fonte: (CAO, L. et al., 2019)

Em (ZHONG, J.; YI, X., 2020), diferentemente dos outros estudos apresentados que utilizaram base dados de apenas um local, existem dados mais dispersos e heterogêneos, uma vez que foram utilizados *EHR* em chinês obtidos de dez hospitais da província chinesa de Shandong. Como ainda não há registros médicos suficientes decidiram utilizar a classificação de categorias considerando apenas as catorze mais populares. Como foi descrito no estudo apresentado anteriormente, a "tokenização" dos textos na língua chinesa são bem diferentes do inglês, que pode ser alcançado através de delimitadores entre os termos, e, por isso, foi utilizado a ferramenta *open source* Han LP para tal finalidade. Uma vez que os métodos mais tradicionais como a *one-hot vector* e a *BoW* possuem limitações em relação a dimensionalidade, esparsidade e informações semânticas das palavras, optou-se por utilizar a técnica *word2vec* para as redes neurais. Como resultado, os estudos experimentais mostraram que o modelo CNN foi o que apresentou melhores resultados comparado aos modelos *RNN*, *LSTM*, *GRU* e *SVM*, um dos algoritmos de *machine learning* tradicionais mais populares que foi escolhido como o *baseline* desse estudo.

No estudo de (BAGHERI, A. et al., 2020) foi utilizado o *dataset* de textos em holandês de altas médicas cardiológicas do Centro Universitário Médico de Utrech. Utilizou-se dois tipos diferentes de predição da CID, uma *single-label* que tem como resultado apenas um código e outra *multi-label* que utiliza a probabilidade de cada código e um *threshold*. Além disso, considerou-se dois níveis diferentes do código internacional: a primeira considerando os vinte e dois capítulos e, na outra, os três primeiros dígitos do código. Foi apresentado um *benchmark* entre três modelos de deep learning (*CNN*, *LSTM* and *BiLSTM* e *HA-GRU*) e dois métodos baseados em *SVM*: um utilizando *BoW* e outro *embeddings* pré-treinados. Chegou-se a conclusão de que a literatura mostra de forma

exagerada os resultados de performance prática e que os métodos de *deep learning* são suficientemente bons para complementar, porém não substituir a codificação CID feita por humanos na aplicação dessa pesquisa.

O artigo que será descrito a seguir, (SAMMANI, A. et al, 2021), é uma continuação de (BAGHERI, A. et al., 2020) e foi focado em atingir resultados que fossem clinicamente úteis. Utilizou-se o mesmo *dataset* em holandês de altas médicas cardiológicas, porém apenas com os códigos que possuem granularidade suficiente para serem clinicamente relevantes como I48, I21 e I42.0. Como na prática os relatórios de alta médica podem possuir mais de uma CID diferente, foi realizada tanto a classificação *single-label*, como também a *multi-label*. Deve-se ressaltar também que foi investigado utilizando o parágrafo de resumo (conclusão), o texto inteiro e, ao final, adicionando as variáveis clínicas gênero e idade. O *pipeline* utilizado foi: fazer o pré-processamento dos texto transformando-os em um vetor *embedding*, fazendo uso das técnicas *DEDUCE* e dos pacotes *tm* e *tidytext* em R, e aplicar modelo *Bidirectional Gated Recurrent Unit Neural Network*. A performance foi muito boa atingindo o *F1-score* de 0,76-0,99 para três caracteres e 0,87-0,98 para quatro caracteres do código CID-10 e ambas foram melhores quando foi utilizado o texto inteiro. As variáveis clínicas não foram relevantes para modificar a performance do modelo. Diferentemente do estudo anterior, dessa vez houve uma preocupação a explicabilidade do modelo, uma vez que não é trivial fazer a explicação de modelos *deep learning*. Foi fornecido ao final o coeficiente das palavras e, para verificar a qualidade da classificação, foi feita uma avaliação manual com médicos especialistas.

Diferentemente de outros estudos que utilizaram apenas o MIMIC-III, em (A. D. Reys et al., 2020), foi utilizado primeiramente textos médicos em português do Brasil para fazer a classificação do CID dos pacientes antes de replicar o modelo para o *dataset* público. Utilizou-se os modelos regressão logística, *CNN* e *CNN-Att* para classificação. É importante destacar também que para a extração de *features* foi utilizado *TF-IDF* para a regressão logística e *word2vec* para as redes neurais. Segundo o estudo, os registros utilizados possuem poucas palavras por documento quando são utilizados apenas resumo de alta médica e, por isso, foram incorporados outros documentos disponíveis para melhorar a performance. O modelo com melhor resultado foi o *CNN-Att* em ambos conjunto de dados com F1 0,537 no MIMIC-III e 0,485 no *dataset* brasileiro com os documentos adicionais.

Em (FERRÃO, J. C. et al., 2013) foi proposto uma metodologia para auxiliar os profissionais codificadores a classificar registros médicos eletrônicos de acordo com a classificação CID-9. Esse estudo utilizou dados com um período de nove meses de um hospital localizado na cidade de Lisboa em Portugal e como havia uma grande quantidade de variáveis disponíveis no *dataset*, foi realizado uma *feature selection* utilizando a técnica *FCBF*. Além disso, um dos objetivos do estudo é superar os desafios em contextos em que o uso de *NLP* se mostra limitado e por isso ele fez uso apenas de dados estruturados.

Para predição foi criado um modelo supervisionado baseado em *SVM* e o resultado do classificador foi um *F1-score* de 52% e a métrica *Recall* foi ligeiramente superior a *Precision* o que é um ótimo resultado para um algoritmo de suporte.

Na pesquisa de (DUARTE, F. et al., 2018) foi desenvolvido uma rede neural que combina *word embeddings*, dois níveis de *recurrent units* e mecanismos de *neural attention* para geração de representação dos textos. Um ponto a se destacar do estudo é que ele mostra a vantagem de se implementar mecanismos de *neural attention* para interpretabilidade dos resultados, destacando os elementos dos dados de entrada que possuem maior contribuição na predição. Para a implementação do modelo foi utilizado o *keras*, biblioteca de *deep learning*, e para algumas tarefas específicas o *scikit-learn*. Os textos utilizados como entrada estão na língua portuguesa e foram extraídos de certificados de óbitos, relatórios de autópsia e boletins clínicos do Ministério Português de Saúde. Eles são referentes ao sistema SICO, que utiliza o CID-10 de acordo com as normas vigentes do governo de Portugal, e foram extraídos dados entre os anos de 2013 e 2015, excluindo mortes neonatal e perinatal. Para avaliar de fato a performance um segundo *dataset* do ano de 2016 foi utilizado, uma vez que este ainda estava sob avaliação de codificadores especialistas no momento em que estava sendo realizado esse estudo. Por fim, o resultado do modelo apresentou precisão de 89%, 81% e 76% para capítulo, bloco e código completo de acordo com o CID-10.

1.2 Objetivo e motivação

A classificação de acordo com os códigos CID tem como objetivo identificar precisamente o diagnóstico de um paciente que foi até o médico para tratar de algum tipo de enfermidade. A tarefa é bastante desafiadora, uma vez que a classificação é muito complexa inclusive para médicos mais experientes no assunto, porém muito importante para os hospitais. Essa classificação é *multi-class* e *multi-label* que sem treinamento profissional, os casos codificados pelos médicos tendem a ser incorretos gerando grandes perdas para os serviços de saúde. Analisando a literatura observa-se que há diferentes técnicas de *NLP* utilizadas, como BoW, *TF-IDF* e *word2vec*, e há diferenças principalmente nos métodos de "tokenização" utilizados devido aos estudos utilizarem textos em diferentes línguas. Por conta disso, existe uma limitação em relação à qualidade dos pacotes de *NLP* em diferentes línguas, existem algumas que são mais avançadas que outras, sem contar que está sendo proposto um estudo na área médica que possui diversas palavras específicas e muitas abreviações. Os métodos tradicionais mais utilizados foram a regressão logística e *SVM*, ao passo que nos estudos mais recentes está sendo utilizado diferentes metodologias derivadas das redes neurais que é o estado-da-arte nesse assunto até o momento. É importante ressaltar também que em todas as bases de dados dos estudos existe um desbalanceamento na quantidade de códigos CID prescritos, os códigos mais frequentes, possuem mais amostras.

Por esse motivo, existem abordagens distintas para superar essa limitação, onde a mais empregada é utilizar apenas os cinquenta códigos CID mais frequentes.

Como pode-se observar fazendo uma busca nos materiais mais relevantes, a grande maioria das pesquisas são recentes e existem poucas referências que utilizam textos na língua portuguesa brasileira, revelando que ainda há um leque bem grande possibilidades a serem exploradas para evolução no campo da inteligência artificial na área da saúde, principalmente no Brasil. Sendo assim, surgiu a motivação desse trabalho de conclusão de curso, cujo objetivo será desenvolver um modelo de classificação utilizando técnicas de *NLP* e *machine learning* já existentes na literatura, com o intuito de auxiliar os profissionais da saúde responsáveis por essa árdua e complexa classificação, reduzindo, assim, o tempo de pesquisa para essa definição. Como essa tarefa é muito complexa, o problema será transformado apenas em multi-classe utilizando apenas os 10 códigos de entrada mais frequentes considerando apenas os três primeiros dígitos que representam a categoria da enfermidade do paciente. Além disso, o poder de processamento disponíveis são escassos, optou-se por utilizar apenas alguns métodos tradicionais não só para extração de *features* do texto, como também para os classificadores. Será utilizada a técnica de NLP *TF-IDF* e os modelos de regressão logística, como *base line*, e SVM que mostra-se muito eficaz em espaços de alta dimensionalidade que, no caso, são as *features* resultantes da metodologia de NLP escolhida. O objetivo principal será criar um pipeline completo de um trabalho de ciência de dados e avaliar se é possível obter um bom classificador utilizando técnicas tradicionais para o problema menos complexo de classificação de código CID que foi proposto.

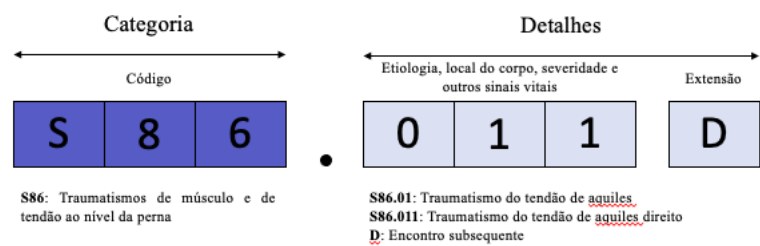
2 FUNDAMENTOS TEÓRICOS

Neste capítulo serão apresentados alguns fundamentos básicos para o entendimento da metodologia proposta neste trabalho.

2.1 Código CID

Como foi descrito no Capítulo 1 ele é uma classificação internacional criada com o objetivo de padronizar e catalogar doenças e problemas relacionados com a saúde. A figura 1 mostra um exemplo de como um código foi definido. Ele é composto de sete caracteres em que o primeiro é obrigatoriamente uma letra e os outros podem ser letras ou números. Os três primeiros indicam a categoria, enquanto os três seguintes a etiologia, local do corpo, severidade e outros sinais vitais. Por fim o sétimo e último caractere, também conhecido como sétimo caractere de extensão, é utilizado em algumas categorias para documentar em que momento foi encontrado o problema de saúde.

Figura 1 – Estrutura do código CID



Fonte: Adaptado de (ESLAMI, S.; ADORJAN, P.; MEINEL, C. Sehmich, 2020)

Na tabela abaixo encontra-se um a hierarquia de classificação da CID-10 resumida, sem os detalhamentos, para entender como ele é dividido.

Tabela 2 – CID-10

Capítulo	Códigos	Título
I	A00-B99	Algumas doenças infecciosas e parasitárias.
II	C00-D48	Neoplasmas (tumores).
III	D50-D89	Doenças do sangue e dos órgãos hematopoéticos e alguns transtornos imunitários.
IV	E00-E90	Doenças endócrinas, nutricionais e metabólicas.
V	F00-F99	Transtornos mentais e comportamentais.
VI	G00-G99	Doenças do sistema nervoso.
VII	H00-H59	Doenças do olho e anexos.
VIII	H60-H95	Doenças do ouvido e da apófise mastoide.
IX	I00-I99	Doenças do aparelho circulatório.
X	J00-J99	Doenças do aparelho respiratório.
XI	K00-K93	Doenças do aparelho digestivo.
XII	L00-L99	Doenças da pele e do tecido subcutâneo.
XIII	M00-M99	Doenças do sistema osteomuscular e do tecido conjuntivo.
XIV	N00-N99	Doenças do aparelho geniturinário.
XV	O00-O99	Gravidez, parto e puerpério.
XVI	P00-P96	Algumas afecções originadas no período perinatal.
XVII	Q00-Q99	Malformações congênitas, deformidades e anomalias cromossômicas.
XVIII	R00-R99	Sintomas, sinais e achados anormais de exames clínicos e de laboratório, não classificados em outra parte
XIX	S00-T98	Lesões, envenenamentos e algumas outras consequências de causas externas.
XX	V01-Y98	Causas externas de morbidade e de mortalidade.
XXI	Z00-Z99	Fatores que influenciam o estado de saúde e o contato com os serviços de saúde.
XXII	U00-U99	Códigos para propósitos especiais.

2.2 Principais técnicas de preprocessamento de textos

- Tokenização: Separação do texto em porções menores, também conhecido como *tokens*. Um texto pode ser dividido em uma sentença, uma sentença em palavras, uma palavra em caracteres. O tipo de *token* depende do tipo de aplicação. Além disso, no processo de tokenização, as pontuações e caracteres especiais costumam ser removidos;
- *Stop words*: Palavras que devem ser ignoradas, uma vez que não possuem importância para o modelo. Existem listas já pré-definidas em pacotes, porém também é possível criar uma lista de *stop words* customizada;
- Lematização: Técnica que extrai a origem da palavra removendo todos os afixos, prefixos e sufixos;

- *Stemming*: Em muitas palavras apenas retirar os afixos não é o suficiente para se obter a origem da palavra. Utilizando essa técnica é possível obter como resultado a palavra de origem, *lemma*. Leva em consideração o significado e morfologia da palavra;

2.3 *Bag of Words*

Essa é uma técnica muito simples e popular para transformar as palavras em uma representação de vetores que fazem com que a máquina consiga entender elas. Na *BoW* é criado um vetor esparsos contendo as palavras únicas presentes em um vocabulário em que cada coluna representa uma dessas palavras. A representação vetorial das sentenças ficaria como o indicado nas Tabelas 3 e 4 onde o valor da posição da palavra no vetor pode ser representada de duas formas: binário, onde é igual a 1 se a palavra está presente e 0 caso contrário, ou pela frequência, em que o valor atribuído é a quantidade de vezes que a palavra aparece no texto. A intuição por trás dessa técnica está no fato de que sentenças semelhantes possuem um conjunto de palavras semelhantes.

Corpus:

- Texto 1: "Praticar esporte é divertido. Esporte é vida."
- Texto 2: "Esse esporte não é divertido."
- Texto 3: "Esse esporte é cansativo."

Tabela 3 – Representação binária

	praticar	esporte	é	divertido	vida	esse	não	cansativo
Texto 1	1	1	1	1	1	0	0	0
Texto 2	0	1	1	1	0	1	1	0
Texto 3	0	1	1	0	0	1	0	1

Fonte: Elaborado pelo autor.

Tabela 4 – Representação pela frequência

	praticar	esporte	é	divertido	vida	esse	não	cansativo
Texto 1	1	2	2	1	1	0	0	0
Texto 2	0	1	1	1	0	1	1	0
Texto 3	0	1	1	0	0	1	0	1

Fonte: Elaborado pelo autor.

2.4 Term Frequency - Inverse Document Frequency

A técnica *TF-IDF* é baseada na *Bag of Words*, *BoW*, porém ela leva em consideração a importância da palavra no documento e no conjunto de todos os documentos. Dessa forma, é calculado uma pontuação para indicar a relevância de cada palavra em cada documento. A importância da palavra no documento é proporcional ao número de vezes em que ela aparece nele (*term frequency*) e inversamente proporcional a quantidade de documentos que possuem essa palavra (*inverse document frequency*), assim o peso dos termos que ocorrem mais frequentemente no conjunto de documentos é menor, ao passo que o peso daqueles que ocorrem raramente é maior.

O cálculo é realizado utilizando as equações 2.1, 2.2 e 2.3 para um dado termo t , documento d e *corpus* de documentos D :

$$tf(t, d) = \frac{f_d(t)}{\max_{w \in d} f_d(w)} \quad (2.1)$$

$$idf = \ln\left(\frac{|D|}{|\{d \in D : t \in d\}|} + 1\right) \quad (2.2)$$

$$tf - idf(t, d) = tf \cdot idf \quad (2.3)$$

Em que: *Corpus*:

- $f_d(t)$: Número de repetições da palavra no documento;
- $\max_{w \in d} f_d(w)$: Número total de palavras no documento;
- $|D|$: Número de documentos;
- $|\{d \in D : t \in d\}|$: Número de documentos contendo a palavra;

2.5 Regressão Logística

A regressão logística, é uma técnica que ajusta de um modelo para melhor classificação de uma variável dicotômica. Ela busca encontrar o hiperplano que melhor separa as duas classes em questão. Ela pode assumir valores entre 0 ou 1 que indica a probabilidade da classe ser igual a 1 e pode ser definida segundo a seguinte equação:

$$\hat{y} = \sigma(w^T \cdot x + b) \quad (2.4)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.5)$$

Em que:

- σ : Função sigmoide;
- w : Vetor de pesos;
- b : Termo de viés, também conhecido como o intercepto;
- x : *features* ou variável explicativa

O vetor de pesos define a importância de cada *feature* para classificação e é definido na fase de treinamento. O *score* z é um valor que indica o quanto o conjunto de *features* são representativas para ser classificado como classe 1 após serem multiplicadas pelos pesos w e pelo intercepto b . Por fim, a função sigmoide tem como objetivo transformar z em probabilidade.

Assim, uma possível fronteira de decisão fica definida da seguinte maneira:

$$Classe = \begin{cases} 1, & \text{se } \hat{y} > 0,5 \\ 0, & \text{caso contrário} \end{cases} \quad (2.6)$$

2.5.1 Regressão Logística Multinomial

Originalmente a regressão logística é limitado a problemas de apenas duas classes. É possível superar esse problema utilizando as técnicas *One-vs-One* ou a *One-vs-Rest* que será descrito na seção 2.7 em que divide-se esse problema de classificação multi-classes em vários problemas de classificação binária. Uma outra alternativa que pode ser utilizada é a regressão logística multinomial em que é utilizado a função *softmax*, equação 2.7, ao invés de utilizar a função *sigmoide*. Dessa forma, a predição torna-se um mapeamento de cada classe dentro de uma função de probabilidade em que cada valor varia entre $[0,1]$, sendo a soma de todos os valores igual a 1.

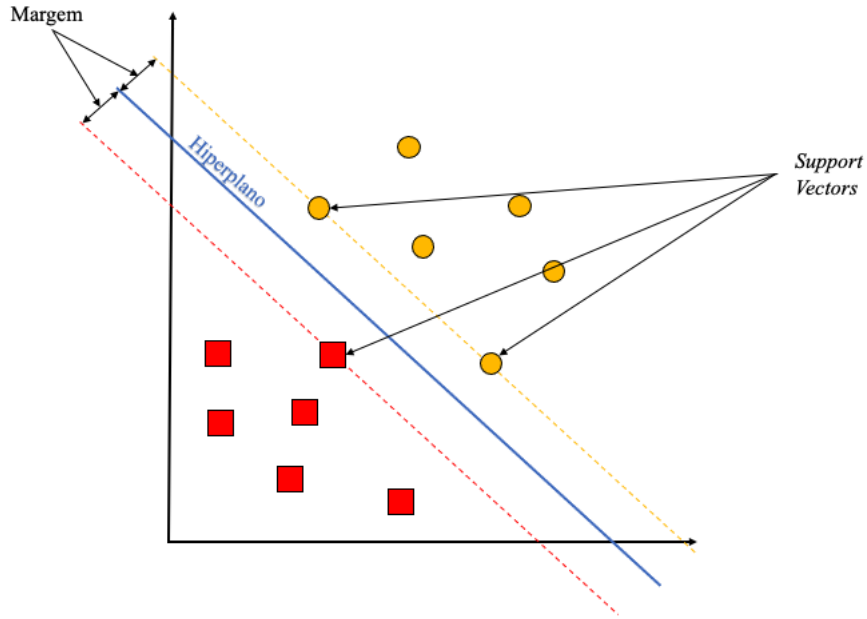
$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}, 1 \leq i \leq k \quad (2.7)$$

2.6 Support Vector Machine

Assim como a regressão logística, a *SVM* busca encontrar a fronteira de separação que divide duas classes e originalmente também é um classificador binário. O algoritmo utiliza funções *kernel* e faz parte dos Classificadores da Margem Máxima, ou seja, ele procura pelo hiperplano a partir da maximização da margem, a fim de criar um espaço ideal para fazer a separação entre as duas classes com segurança. A margem é o espaço vazio que fica entre o hiperplano e os pontos mais externos de cada grupo, os *support*

vectors. Eles são os pontos de cada classe mais próximos do hiperplano e são eles que vão influenciar diretamente na posição e orientação deste.

Figura 2 – Representação espacial *SVM*

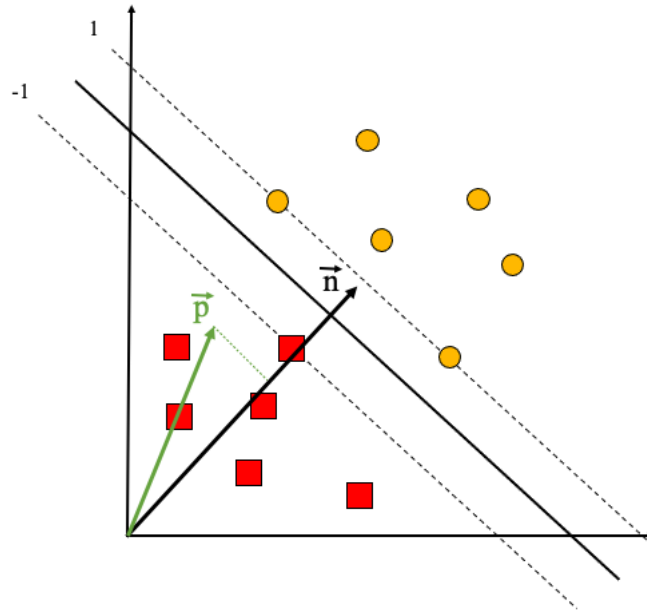


Fonte: Elaborado pelo autor.

Para explicar maximização das margens suponha um vetor n normal à fronteira de decisão e outro vetor p que indica a posição em relação à origem de um determinado valor que deseja-se encontrar a classe a qual pertence. Com o intuito de definir se a classe desse ponto é quadrado ou círculo pode-se calcular a projeção do vetor p no vetor n e observar o tamanho da projeção. Se ela for maior que um determinado valor o objeto pertence a classe círculo, caso seja menor ela será considerada quadrado ou pode estar na margem. Ao considerar que:

$$Classe = \begin{cases} \text{Círculo,} & \text{se } \text{proj}_{\vec{n}} \vec{p} \geq 1 \\ \text{Quadrado,} & \text{se } \text{proj}_{\vec{n}} \vec{p} \leq -1 \end{cases} \quad (2.8)$$

Figura 3 – Calculo da margem máxima



Fonte: Elaborado pelo autor.

Pode-se escrever essa condição matematicamente como:

$$Classe = \begin{cases} \text{Círculo}, & \text{se } w^T \cdot x + b \geq 1 \\ \text{Quadrado}, & \text{se } w^T \cdot x + b \leq -1 \end{cases} \quad (2.9)$$

O hiperplano de decisão pode ser definido em termos de um intercepto b e um vetor normal ao hiperplano de decisão $\vec{w}$ que é perpendicular ao hiperplano (CHRISTOPHER, D. M.; PRABHAKAR, R.; HINRICH, S., 2008). Dessa forma, o classificador linear pode ser escrito em apenas uma equação como:

$$y_i(w_i^T \cdot x + b) \geq 1 \quad (2.10)$$

Em que o $y_i = -1$ indica a classe quadrado e o $y_i = 1$ a classe círculo.

Dessa forma, pode-se calcular o tamanho da margem como sendo a diferença entre a projeção do *support vector* da classe positiva (círculo) e o *support vector* da classe negativa (quadrado). Assim, o problema de maximização pode ser escrito da seguinte maneira:

$$\text{Maximizar } (x_{+sv} - x_{-sv}) \cdot \frac{w}{|w|} \quad (2.11)$$

Ao substituir na equação 2.10, temos que:

$$\text{Maximizar } \frac{2}{|w|} \quad (2.12)$$

Que também pode-se escrever com um problema de minimização para:

$$\text{Minimizar } \frac{1}{2}|w|^2 \quad (2.13)$$

Em que também deve ser satisfeita a restrição para todos os pontos i :

$$y_i(w_i^T \cdot x + b) \geq 1 \quad (2.14)$$

As funções *kernel* são utilizadas para problemas que não são linearmente separáveis. Por meio delas é possível projetar os pontos originais em um espaço de dimensão maior fazendo com que eles sejam linearmente separáveis. Essa manipulação não faz essa projeção de fato, ao invés de realizar o produto escalar entre dois vetores de alta dimensionalidade que produz um número escalar, é possível chegar a esse valor utilizando uma função *kernel*. A grande vantagem das funções kernel é que além de ajudar em problemas linearmente não separáveis elas diminuem o custo computacional. Na tabela 5 abaixo encontram-se algumas das funções mais utilizadas.

Tabela 5 – Funções Kernel

Função Kernel	Expressão	Hiperparâmetros
Linear	$K(x_i, x_j) = x_i \cdot x_j$	
Polinomial	$K(x_i, x_j) = (\gamma(x_i \cdot x_j) + r)^d$	$\gamma; d; r$
RBF	$K(x_i, x_j) = e^{(-\gamma\ x_i - x_j\ ^2)}$	γ
Sigmoid	$K(x_i, x_j) = \tanh(\gamma(x_i \cdot x_j) + r)$	$\gamma; r$

Fonte: Elaborado pelo autor.

2.7 Técnicas de classificação para problemas multi-classe

Muitas técnicas de classificação são originalmente binárias e, por isso, são limitadas a problemas com apenas duas classes. Duas maneiras de superar essa característica limitante para gerar classificadores com várias classes é utilizando as técnicas *One-vs-Rest* e *One-vs-One* que serão apresentadas nessa seção. Ambas dividem problemas multi-classes em problemas binários a fim de superar essa limitação.

2.7.1 *One-vs-Rest*

Também conhecido como *One-vs-All*, é a técnica mais comum e geralmente é utilizado em modelos de regressão logística. Nela é induzido um classificador para cada

classe, assim ajusta-se o modelo comparando a classe em questão contra as outras. Dessa forma, são construídos n classificadores em que n é o número de rótulos do problema.

Uma situação, por exemplo, em que se deseja classificar objetos 2D de acordo com quatro classes possíveis de formas: quadrado, losango, triângulo e retângulo, pode ser dividido em quatro problemas binomiais:

- quadrado contra [losango, triângulo, retângulo]
- losango contra [quadrado, triângulo, retângulo]
- triângulo contra [quadrado, losango, retângulo]
- retângulo contra [quadrado, losango, triângulo]

2.8 Métricas de avaliação de modelos

Nessa seção serão apresentados as métricas que serão utilizadas para avaliação dos modelos. Antes de iniciar é importante explicar a Matriz de Confusão que servirá como base para visualizar o desempenho do modelo e para calcular as métricas.

Para um problema binário ela é uma tabela de duas linhas e duas colunas que indica a quantidade de verdadeiros positivos, falsos positivos, verdadeiros negativos e falsos negativos. Positivo e negativo seriam as classes do problema. A representação é feita conforme a tabela 6 abaixo.

Tabela 6 – Matriz de Confusão

	Valor Predito	
	Positivo	Negativo
Valor Real	VP	FN
	FP	VN

Fonte: Elaborado pelo autor.

Em que:

- VP: Verdadeiro positivo
- FP: Falso positivo
- VN: Verdadeiro negativo
- FN: Falso negativo

Essas mesmas abreviações serão utilizadas para explicar as métricas abaixo.

2.8.1 Acurácia

A acurácia mede a proporção de predições corretas dentre todas as predições.

Binário:

$$Acurácia = \frac{VP + VN}{VP + FP + VN + FN} \quad (2.15)$$

2.8.2 Precisão

Para os valores reais da classe positivo, qual a proporção que foram classificados corretamente.

Binário:

$$Precisão = \frac{VP}{VP + FP} \quad (2.16)$$

2.8.3 Revocação

Para os valores preditos para classe positivo, qual a proporção que foram classificados corretamente.

Binário:

$$Revocação = \frac{VP}{VP + FN} \quad (2.17)$$

2.8.4 *F1-Score*

Média harmônica das métricas Precisão e Revocação e indica quanto o modelo é preciso e robusto, ou seja, se ele acerta bastante e se ele classifica bem todas as classes. Essa métrica é muito importante para avaliar problemas que possuem classes desbalanceadas, se qualquer uma delas for baixa, consequentemente o F1 será baixo também.

Binário:

$$F1-Score = 2 \frac{Precisão \cdot Revocação}{Precisão + Revocação} \quad (2.18)$$

2.8.5 *Grid Search*

Técnica em que é feito a busca exaustiva dos hiperparâmetros com o objetivo de otimizar o modelo. Os hiperparâmetros são características externas do modelo que não é possível ser estimado por meio dos dados. Um exemplo é o k do algoritmo *k-Nearest Neighbors* que indica a quantidade de vizinhos próximos para determinar a classe de um ponto.

Nessa técnica é necessário informar, antes de estimar o modelo, todos os valores dos hiperparâmetros. O passo seguinte é calcular para todas as combinações possíveis as métricas de avaliação do modelo. Por fim, é possível escolher a combinação que traga a melhor performance para o modelo. No *scikit-learn* a função que realiza essa tarefa é a *GridSearchCV*.

3 METODOLOGIA

Esse trabalho será dividido em basicamente seis partes:

1. Extração dos dados;
2. Preprocessamento dos dados;
3. *Feature engineering* dos textos;
4. Construção dos modelos;
5. Avaliação dos modelos;
6. Avaliação dos resultados;

Todo o trabalho será desenvolvido utilizando a linguagem *Python* e as bibliotecas *Scikit-learn*, *Pandas*, *NLTK* e *Plotly*.

A base de dados que será utilizada para a tarefa de classificação dos códigos CID de acordo com os textos médicos será a MIMIC III retirada de (Johnson, A., Pollard, T., and Mark, R., 2016). Ela é uma base de dados muito utilizada em pesquisas na área da saúde, contendo registros estruturados e não estruturados de aproximadamente 46.000 pacientes que ficaram nas unidades de tratamento intensivo do Centro Medico Beth Israel Deaconess entre 2001 e 2012.

Para se ter acesso a essa base de dados é necessário fazer um cadastro em (Johnson, A., Pollard, T., and Mark, R., 2016) e fazer a solicitação junto com o orientador do projeto. Nessa solicitação é necessário informar seus dados e para que fim será utilizado esse *dataset*. Além disso, é necessário fazer um curso relacionado às regras de uso de dados médicos dentro dos EUA. Por fim é possível fazer o download dos dados ou acessá-los por meio do *BigQuery*. Para esse trabalho foi escolhido o *BigQuery*, *data warehouse* do *Google*, em que é possível acessar os dados utilizando a linguagem SQL.

3.1 Extração dos dados

A base de dados MIMIC III está dividido em diversas tabelas em que é possível fazer a correspondência utilizando alguns tipos de *ID's*. Como o objetivo é fazer a classificação de acordo com os textos, o primeiro passo é identificar as tabelas de textos e diagnósticos de acordo com a CID juntamente com as respectivas chaves para fazer a correspondência entre esses dois dados.

Sabe-se que os códigos CID do *dataset* estão no padrão CID-9, dessa forma o passo seguinte será realizar uma conversão desses códigos para o padrão CID-10 para facilitar a interpretação, já que é o padrão atual que está sendo utilizado. Essa etapa será realizada utilizando uma tabela de correspondências entre os dois padrões retirado de (*Centers for Medicare and Medicaid Services*, 2018) agência federal do governo norte americano que faz parte do Departamento Norte Americano de Saúde e Serviços Humanos. Após essa conversão será feito um corte no código, extraindo apenas os três primeiros dígitos do código que indicam a categoria da doença.

Por fim, será feito um filtro nos dados. Serão utilizados apenas os textos de resumos de altas médicas e apenas as 10 categorias de entrada do caso com mais registros.

3.2 Preprocessamento dos dados

Sabe-se que nos textos do MIMIC III, os dados anonimizados estão entre dois asteriscos (** exemplo **). Dessa forma, todos os textos que estiverem nesse padrão serão eliminados.

Outros preprocessamentos que serão utilizados são: retirar caracteres especiais, números e palavras com duas ou menos letras.

A última etapa será realizar a separação dos dados entre treino (70%) e teste (30%) estratificada para cada conjunto de dados, já que os códigos CID são desbalanceados.

3.3 Feature engineering do textos

Com os dados-não estruturados transformados em vetores, será realizado a *feature engineering TF-IDF* que servirá como entrada para os modelos. Nesses processos, será feito a "*Tokenização*" e remoção de *Stop Words*. A Lematização e o *Stemming* serão tratados ao final apenas no modelo que possuir melhor resultado com o intuito de verificar se essas técnicas de processamento de linguagem natural farão diferença nas métricas do modelo.

3.4 Construção dos modelos

Tanto a regressão logística quanto o *SVM* são originalmente binários, dessa maneira eles serão utilizados em conjunto apenas com a técnica *One-vs-Rest*, que é computacionalmente menos custosa e já está implementado por padrão no *scikit-learn*, para torná-los multi-classe. Para a regressão logística será avaliado também a utilização da função *softmax*, seção 2.5.1,

No caso do *SVM* será testado apenas a função *kernel* linear.

3.5 Avaliação dos modelos

Todos os modelos serão avaliados segundo as métricas de Acurácia, *Precision*, *Recall*, *F1-Score* que estão descritos no capítulo 2.

3.6 Avaliação dos resultados

Ao final serão obtidos modelos de regressão logística e *SVM*, mas que totalizarão 5 *pipelines* possíveis resultado da combinação de pré-processamento/*feature engineering*, algoritmos e técnicas de transformação problemas binários em multi-classe.

O passo seguinte será realizar um *Grid Search* na melhor combinação de técnicas/algoritmos para a tarefa proposta e obter os melhores hiperparâmetros com o objetivo de tornar o modelo de classificação mais acurado.

Por fim, serão extraídas as dez *features* de maior importância para cada classe, no caso, cada código CID, para avaliar a interpretabilidade do melhor resultado. Dessa forma, será possível avaliar quais palavras foram mais relevantes para a classificação e observar se elas realmente fazem sentido para essa tarefa.

4 RESULTADOS DO TRABALHO

4.1 Extração da base de dados

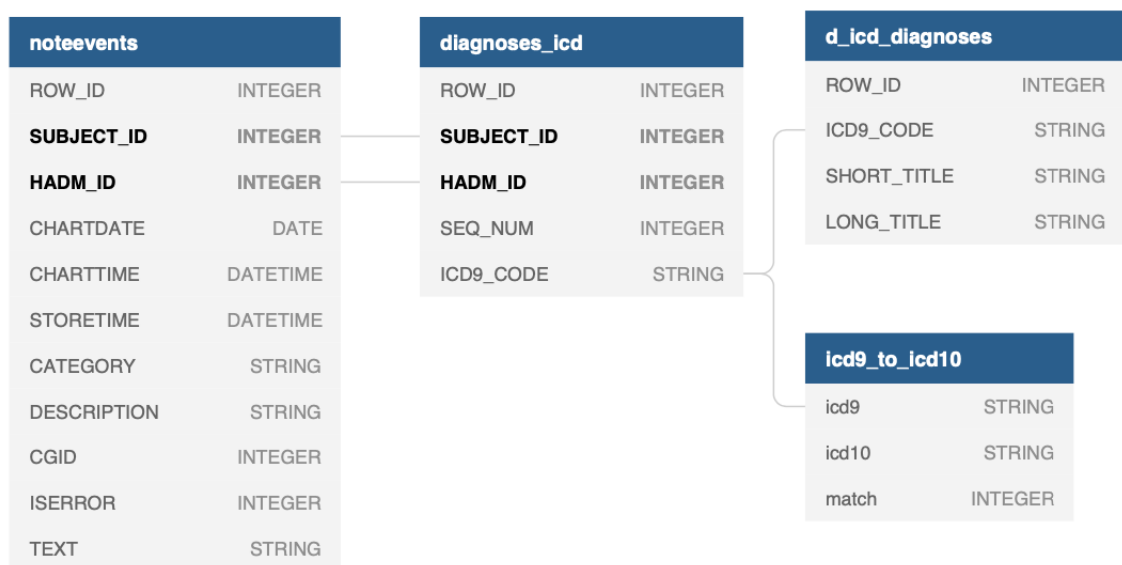
As tabelas utilizadas do MIMIC III foram:

- `physionet-data.mimiciii_notes.noteevents`: Possui os textos médicos (TEXT) em que podem ser filtrados apenas os resumos de alta médica pela coluna CATEGORY.
- `physionet-data.mimiciii_clinical.diagnoses_icd`: Possui o CID-9 completo (ICD9_CODE) e a ordem cronológica de cada código que foi estabelecido para a internação (SEQ_NUM).
- `physionet-data.mimiciii_clinical.d_icd_diagnoses`: Possui a descrição do código CID-9 (SHORT_TITLE e LONG_TITLE).

As *keys* SUBJECT_ID e HADM_ID indicam o ID do paciente e o ID de admissão nas unidades de tratamento intensivo respectivamente.

A tabela `icd9_to_icd10` é a relação entre os dois padrões CID que foi retirada de (Centers for Medicare and Medicaid Services, 2018) e posteriormente adicionada no ambiente do *BigQuery*.

Figura 4 – Schema e conexões das tabelas utilizadas no *BigQuery*



Fonte: Elaborado pelo autor.

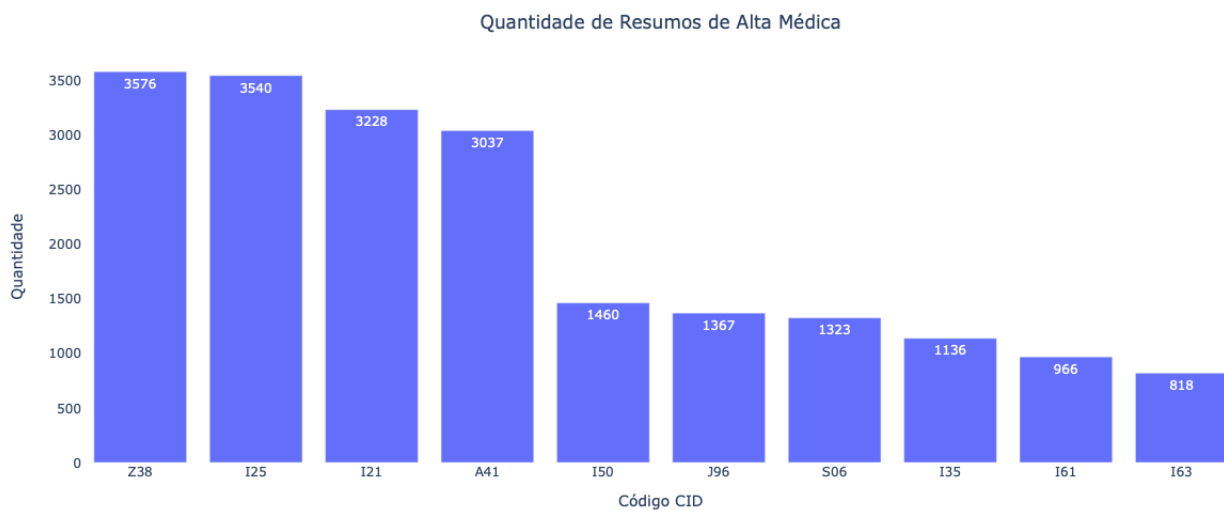
A base final ficou composta pelas colunas e apenas as dez categorias do CID-10 com mais registros foram selecionadas:

- SUBHADM_ID: Chave concatenada SUBJECT_ID e HADM_ID
- ICD9_CODE: Código CID-9 completo
- ICD10_CAT: Código CID-10 contendo apenas a categoria
- LONG_TITLE: Título da enfermidade do CID-9
- TEXT: Resumos de altas médicas

4.2 Análise de Dados

Ao considerar apenas os resumos de altas médicas e a categoria segundo a CID-10, os dez códigos com mais registros e a frequência deles encontram-se na figura 5.

Figura 5 – Pareto da quantidade dos 10 códigos mais frequentes.



Fonte: Elaborado pelo autor.

- Z38: Nascidos vivos (nati-vivos) segundo o local de nascimento
- I25: Doença isquêmica crônica do coração
- I21: Infarto agudo do miocárdio
- A41: Outras septicemias
- I50: Insuficiência cardíaca
- J96: Insuficiência respiratória não classificada de outra parte

- S06: Traumatismo intracraniano
- I35: Transtornos não-reumáticos da valva aórtica
- I61: Hemorragia intracerebral
- I63: Infarto cerebral

Nota-se que mesmo considerando apenas a categoria, os dados são bem desbalanceados e há uma predominância de códigos do capítulo de Doenças do aparelho circulatório.

4.3 Avaliação dos Resultados

4.3.1 Modelos

Na tabela 7 encontra-se os resultados das métricas dos modelos regressão logística, regressão logística com *softmax* e SVM com *kernel* linear que tiveram como *feature engineering* o *TF-IDF*:

Tabela 7 – Resultados dos modelos

	Acurácia	Precisão	Revocação	<i>F1-Score</i>
Regressão Logística	84,43%	85,62%	85,67%	85,47%
Regressão Logística + <i>Softmax</i>	85,02%	86,24%	86,31%	86,17%
SVM <i>Kernel</i> Linear	86,11%	86,77%	86,88%	86,76%

Fonte: Elaborado pelo autor.

Observa-se que em todas as métricas o SVM com *kernel* linear superou o modelo *base line* da regressão logística.

4.3.2 Lemmatização e *Stemming* no melhor modelo

Na tabela 8 encontra-se os resultados das métricas do melhor modelo encontrado, svm com *kernel* linear, que tiveram como *feature engineering* o *TF-IDF* adicionando separadamente a lematização e o *stemming*:

Tabela 8 – Resultados dos modelos com Lemmatização e *Stemming*

	Acurácia	Precisão	Revocação	<i>F1-Score</i>
SVM <i>Kernel</i> Linear + Lemmatização	86,11%	86,81%	86,91%	86,87%
SVM <i>Kernel</i> Linear + <i>Stemming</i>	86,24%	86,93%	87,03%	86,92%

Fonte: Elaborado pelo autor.

Os dois modelos, SVM com *kernel* linear com lematização e com *stemming* foram superiores ao sem nenhum desses tratamentos, porém como foi necessário uma

implementação a parte adicionando uma função na "tokenização" do corpus, já que não é possível realizar nativamente no *scikit-learn*, o tempo de execução foi muito maior do que o modelo SVM com *kernel* linear sem *stemming* sem acarretar em grandes mudanças nos resultados. Dessa forma, o melhor modelo para ser colocado em produção seria o SVM com *kernel* linear.

4.3.3 Grid Search

As opções escolha do hiperparâmetro C, parâmetro de regularização, que influência no tamanho da margem para o SVM com *kernel* linear a serem investigadas foram extraídas de (VIRGINIO, L.; REIS, J. C. dos., 2018). O outro hiperparâmetros escolhidos do SVM foi *class_weight* que ajusta automaticamente os pesos inversamente proporcional a frequência das classes. Além disso, para a *feature engineering* dos textos utilizando o TF-IDF, foram investigados *ngram_range*, tamanho dos *n-grams*, e *max_df*, ignora termos que possuem uma frequência nos documentos maior que o threshold indicado. Exceto o C, os outros hiperparâmetros foram escolha do autor. A tabela abaixo mostra o grid de todos os hiperparâmetros utilizados.

Tabela 9 – Opções do *Grid Search*

	Opção 1	Opção 2	Opção 3	Opção 4	Opção 5
C	0.02	0.2	1	2	4
class_weight	balanced	None	-	-	-
max_df	0.7	0.8	0.9	1	-
ngram_range	(1, 1)	(1, 2)	-	-	-

Fonte: Elaborado pelo autor.

Ao final do *Grid Search*, os melhores parâmetros encontrados e os resultados foram:

- C: 2
- class_weight: None
- max_df: 0.8
- ngram_range: (1, 2)

Tabela 10 – Melhor Modelo com *Grid Search*

	Acurácia	Precisão	Revocação	F1-Score
SVM <i>Kernel</i> Linear + <i>Grid Search</i>	88,26%	88,34%	88,26%	88,07%

Fonte: Elaborado pelo autor.

4.3.4 Interpretação dos resultados

Além de avaliar as métricas de um modelo, quando possível, é de extrema importância avaliar as *features* que nesse caso são as palavras que mais contribuíram para a classificação. A tabela 11 contém as dez palavras mais relevantes para a classificação de cada classe.

Tabela 11 – Dez palavras mais relevantes para classificação de cada CID

CID	1	2	3	4	5	6	7	8	9	10
A41	sepsis	septic	hypotension	lactate	septic shock	levophed	shock	ivf	urosepsis	abscess
I21	ntemi	troponin	stemi	elevation	myocardial infarction	myocardial	plavix	heparin	elevations	elevation myocardial
I25	angina	stress	cabg	stress test	elective	referred	ett	test	artery disease	catheterization
I35	aortic	avr	aortic stenosis	valve	valve replacement	replacement	aortic valve	corevalve	severe aortic	diagnosis aortic
I50	chf	heart failure	lasix	diuresis	chf exacerbation	dyspnea	sob	cardiomyopathy	congestive heart	exacerbation
I61	hemorrhage	bleed	headache	thalamia	drain	cerebellar hemorrhage	iph	intraparenchymal	ich	ganglia
I63	stroke	infarct	mca	tpa	carotid	neurology	embolic	infarction	right	infarcts
J96	respiratory failure	respiratory	intubation	intubated	bipap	bronchoscopy	failure	breathing	solumedrol	micu
S06	subdural	fall	subarachnoid	subdural hematoma	trauma	head	neurosurgery	contusion	subarachnoid hemorrhage	sah
Z38	infant	baby	delivery	mother	newborn	nicu	pregnancy	fetal	prenatal	neonatal

Fonte: Elaborado pelo autor.

Como pode ser observado na tabela acima, foi possível obter *features* muito boas para essa tarefa de classificação. Como exemplo pode-se analisar o CID Z38 (Nascidos vivos (nati-vivos) segundo o local de nascimento) que possuem palavras ligadas a recém nascidos, como bebê, mãe, infantil e nascer.

Além disso, mesmo com seis CIDs do capítulo I relacionado a doenças do sistema circulatório, foi possível obter um bom resultado de classificação.

5 CONCLUSÕES

O método desenvolvido neste trabalho mostrou-se ser simples, eficiente e facilmente interpretável. Como foi detalhado anteriormente, a classificação completa dos códigos CID é muito complexa e desafiadora, pela quantidade de possibilidades, códigos com várias hierarquias e por ser um problema *multi-label* e multi-classe, apesar disso esse trabalho mostrou ser possível obter bons resultados diminuindo a complexidade da classificação utilizando técnicas simples indicando a possibilidade de criação de uma ferramenta que consiga realizar a classificação completa dos códigos utilizando técnicas mais recentes e avançadas com o objetivo de auxiliar os médicos responsáveis por essa classificação.

Nesse trabalho foi possível atingir o objetivo inicial, desenvolver um conjunto de métodos completo desde a extração dos dados até a interpretação do modelo e obter resultados satisfatórios, contemplando um estudo, então, em ciência de dados. O melhor modelo encontrado foi o SVM com *kernel* linear em que foi realizado um *Grid Search* em alguns hiperparâmetros obtendo as métricas na casa dos 88%. Ele superou ao menos 2,5% cada métrica analisada em comparação ao *base line*, a regressão logística que já apresentou resultados bastante interessantes.

Alguns aspectos que não foram abordados nessa metodologia que poderiam ser desenvolvidos em trabalhos futuros seriam:

- Aplicar outros algoritmos tradicionais de classificação como *Naive Bayes* e *Random Forest* e avaliar os resultados.
- Aplicar métodos de redes neurais que são o estado da arte tanto na parte de processamento de linguagem natural, quanto na de aprendizado de máquina.
- Aumentar o número de classes para ver como o classificador se comporta.
- Aplicar em registros médicos em português.

6 APÊNDICE

```
# -*- coding: utf-8 -*-
"""TCC.ipynb
```

Automatically generated by Colaboratory.

Original file is located at

→ https://colab.research.google.com/drive/1FkcZBurhsy0AAzmm7nLvhXn8Nk0tm_RK

```
# Import Libraries
```

```
# Pandas and Numpy
```

```
import pandas as pd
import numpy as np
```

```
# NLP
```

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('wordnet')
```

```
# Plotly
```

```
import plotly.express as px
```

```
# Sklearn
```

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.svm import SVC
```

```
from sklearn.metrics import balanced_accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV

def metrics(y_test,y_pred):
    acc = balanced_accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average="weighted")
    recall = recall_score(y_test, y_pred, average="weighted")
    f1 = f1_score(y_test, y_pred, average="weighted")

    print(f"Acurácia: {acc}")
    print(f"Precision: {precision}")
    print(f"Recall: {recall}")
    print(f"F1-Score: {f1}")

from google.colab import drive
drive.mount('/content/drive')

df_mimic = pd.read_json("/content/drive/MyDrive/Colab
↳ Notebooks/bq-results-20220118-214022-y6ditlgl7bb5.json", lines=True)
df_mimic

# Remover caracteres especiais, números e palavras com menos de dois
↳ caracteres
df_mimic["TEXT"] = df_mimic["TEXT"].replace(r'[^a-zA-Z]+', ' ',
↳ regex=True)
df_mimic["TEXT"] = df_mimic["TEXT"].replace(r'\b\w{1,2}\b', ' ',
↳ regex=True)

# Quantidade de registros de cada código CID
icd_qty = df_mimic["ICD10_CAT"].value_counts()
icd_qty

# Gráfico de pareto com as quantidades de cada classe
fig = px.bar(x=icd_qty.index,y=icd_qty,text=icd_qty)
fig.update_layout(title = {
```

```

        "text": "Quantidade de Resumos de Alta
        ↪ Médica",
        "y":0.95,
        "x":0.5,
        "xanchor": "center",
        "yanchor": "top"
    },
    xaxis_title = "Código CID",
    yaxis_title = "Quantidade",
    plot_bgcolor = "rgba(0,0,0,0)")

fig.show()

# Variaveis preditoras e variáveis respostas
X = df_mimic["TEXT"].values
y = df_mimic["ICD10_CAT"].values

# Train test split stratificado
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    ↪ random_state=12, stratify=y)

"""# TF-IDF"""

# Pipeline Regressão Logística
text_clf_tfidf_lr = Pipeline([("tfidf",
    ↪ TfidfVectorizer(stop_words="english")),
    ("clf_lr", LogisticRegression(penalty='l2',
    ↪ multi_class="ovr", random_state=12)),
])

# Fit transform
text_clf_tfidf_lr = text_clf_tfidf_lr.fit(X_train,y_train)

# Predição
y_predicted_tfidf_lr = text_clf_tfidf_lr.predict(X_test)

# Avaliação
metrics(y_test, y_predicted_tfidf_lr)

# Pipeline Regressão Logística Softmax

```

```
text_clf_tfidf_lr_softmax = Pipeline([("tfidf",
    ↪ TfidfVectorizer(stop_words="english")),
    ("clf-lr-softmax",
    ↪ LogisticRegression(penalty='l2',
    ↪ multi_class="multinomial",
    ↪ max_iter=200,
    ↪ random_state=12)),
])
```

```
# Fit transform
```

```
text_clf_tfidf_lr_softmax =
    ↪ text_clf_tfidf_lr_softmax.fit(X_train,y_train)
```

```
# Predição
```

```
y_predicted_tfidf_lr_softmax = text_clf_tfidf_lr_softmax.predict(X_test)
```

```
# Avaliação
```

```
metrics(y_test, y_predicted_tfidf_lr_softmax)
```

```
# Pipeline SVM
```

```
text_clf_tfidf_svm = Pipeline([('tfidf',
    ↪ TfidfVectorizer(stop_words="english")),
    ('clf-svm', LinearSVC(penalty='l2',
    ↪ multi_class="ovr", random_state=12)),
])
```

```
# Fit transform
```

```
text_clf_tfidf_svm = text_clf_tfidf_svm.fit(X_train,y_train)
```

```
# Predição
```

```
y_predicted_tfidf_svm = text_clf_tfidf_svm.predict(X_test)
```

```
# Avaliação
```

```
metrics(y_test, y_predicted_tfidf_svm)
```

```
"""# Adicionando Lemmatization"""
```

```
class LemmaTokenizer(object):
```

```
    def __init__(self):
```

```

        self.wnl = WordNetLemmatizer()
    def __call__(self, text):
        return [self.wnl.lemmatize(t) for t in word_tokenize(text) if t
        ↪ not in stopwords.words("english")]

# Pipeline SVM e Lemmatization
text_clf_svm_lemma = Pipeline([("tfidf",
    ↪ TfidfVectorizer(tokenizer=LemmaTokenizer())),
    ("clf-svm", LinearSVC(penalty="l2",
    ↪ multi_class="ovr", random_state=12)),
])

# Fit transform
text_clf_svm_lemma = text_clf_svm_lemma.fit(X_train,y_train)

# Predição
y_predicted_svm_lemma = text_clf_svm_lemma.predict(X_test)

# Avaliação
metrics(y_test, y_predicted_svm_lemma)

"""# Adicionando Stemming"""

class StemTokenizer(object):
    def __init__(self):
        self.ps = PorterStemmer()
    def __call__(self, text):
        return [self.ps.stem(t) for t in word_tokenize(text) if t not in
        ↪ stopwords.words("english")]

# Pipeline SVM e Stemming
text_clf_svm_stem = Pipeline([("tfidf",
    ↪ TfidfVectorizer(tokenizer=StemTokenizer())),
    ("clf-svm", LinearSVC(penalty="l2",
    ↪ multi_class="ovr", random_state=12)),
])

# Fit transform
text_clf_svm_stem = text_clf_svm_stem.fit(X_train,y_train)

```

```
# Predição
```

```
y_predicted_svm_stem = text_clf_svm_stem.predict(X_test)
```

```
# Avaliação
```

```
metrics(y_test, y_predicted_svm_stem)
```

```
"""# Grid Search do modelo SVM com kernel linear"""
```

```
text_clf_tfidf_svm = Pipeline([('tfidf',  
    ↪ TfidfVectorizer(stop_words="english")),  
                                ('clf-svm', LinearSVC(penalty='l2',  
    ↪ multi_class="ovr", random_state=12)),  
])
```

```
parameters_svm = {'tfidf__ngram_range': [(1, 1), (1, 2)],  
                  'tfidf__max_df': [0.7, 0.8, 0.9, 1],  
                  'clf-svm__C': [0.02, 0.2, 1.0, 2.0, 4.0],  
                  'clf-svm__class_weight': ['balanced', None]  
                  }
```

```
gs_clf_svm = GridSearchCV(text_clf_tfidf_svm, parameters_svm)  
gs_clf_svm = gs_clf_svm.fit(X_train,y_train)
```

```
gs_clf_svm.best_score_
```

```
gs_clf_svm.best_params_
```

```
gs_clf_svm.cv_results_
```

```
from sklearn.model_selection import StratifiedKFold
```

```
skf = StratifiedKFold(n_splits=5)
```

```
scores = []
```

```
text_clf_tfidf_svm_gs = Pipeline([('tfidf',  
    ↪ TfidfVectorizer(stop_words="english", max_df=0.8,  
    ↪ ngram_range=(1,2))),
```

```

        ('clf-svm', LinearSVC(penalty='l2', C=2,
                               ↪ multi_class="ovr",
                               ↪ class_weight=None,
                               ↪ random_state=12)),
    ])

for train_index, test_index in skf.split(X, y):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]

    # Fit transform
    text_clf_tfidf_svm_gs = text_clf_tfidf_svm_gs.fit(X_train, y_train)

    # Predição
    y_predicted_tfidf_svm_gs = text_clf_tfidf_svm_gs.predict(X_test)
    score = accuracy_score(y_test, y_predicted_tfidf_svm_gs)
    scores.append(score)

scores

"""# Interpretabilidade do modelo SVM com kernel linear"""

# Pipeline SVM
text_clf_tfidf_svm_gs = Pipeline([('tfidf',
    ↪ TfidfVectorizer(stop_words="english", max_df=0.8,
    ↪ ngram_range=(1,2))),
    ('clf-svm', LinearSVC(penalty='l2', C=2,
                           ↪ multi_class="ovr",
                           ↪ class_weight=None,
                           ↪ random_state=12)),
    ])

# Fit transform
text_clf_tfidf_svm_gs = text_clf_tfidf_svm_gs.fit(X_train, y_train)

# Predição
y_predicted_tfidf_svm_gs = text_clf_tfidf_svm_gs.predict(X_test)

# Avaliação

```

```
metrics(y_test, y_predicted_tfidf_svm_gs)
```

```
def print_top10(vectorizer, clf, class_labels):  
    feature_names = vectorizer.get_feature_names_out()  
    for i, class_label in enumerate(class_labels):  
        top10 = reversed(np.argsort(clf.coef_[i])[-10:])  
        print("%s: %s" % (class_label,  
                          ", ".join(feature_names[j] for j in top10)))
```

```
print_top10(text_clf_tfidf_svm_gs.named_steps['tfidf'],  
↪ text_clf_tfidf_svm_gs.named_steps["clf-svm"],  
↪ text_clf_tfidf_svm_gs.classes_)
```

REFERÊNCIAS

- BAGHERI, A. et al. Automatic icd-10 classification of diseases from dutch discharge letters. In: SCITEPRESS. **BIOINFORMATICS 2020-11th International Conference on Bioinformatics Models, Methods and Algorithms, Proceedings; Part of 13th International Joint Conference on Biomedical Engineering Systems and Technologies, BIOSTEC 2020**. [S.l.], 2020. v. 13, p. 281–289.
- CAO, L. et al. Automatic icd code assignment based on icd's hierarchy structure for chinese electronic medical records. **AMIA Summits on Translational Science Proceedings**, American Medical Informatics Association, v. 2019, p. 417, 2019.
- CHRISTOPHER, D. M.; PRABHAKAR, R.; HINRICH, S. **Introduction to information retrieval**. [S.l.]: Cambridge University Press, 2008.
- DUARTE, F. et al. Deep neural models for icd-10 coding of death certificates and autopsy reports in free-text. **Journal of biomedical informatics**, Elsevier, v. 80, p. 64–77, 2018.
- ESLAMI, S.; ADORJAN, P.; MEINEL, C. Sehmie: Semi-hierarchical multi-label icd code classification. In: **CLEF (Working Notes)**. [S.l.: s.n.], 2020.
- FERRÃO, J. C. et al. Using structured ehr data and svm to support icd-9-cm coding. In: IEEE. **2013 ieee international conference on healthcare informatics**. [S.l.], 2013. p. 511–516.
- JOHNSON A., P. T.; MARK, R. **MIMIC-III Clinical Database (version 1.4)**. **PhysioNet**. 2016. Disponível em: <<https://doi.org/10.13026/C2XW26>>.
- MARTINS, R. C. **Avaliação da eficácia do software Iris para uso no Brasil**. 2017. Tese (Doutorado) — Universidade de São Paulo, 2017.
- MIKOLOV, T. et al. Efficient estimation of word representations in vector space. **arXiv preprint arXiv:1301.3781**, 2013.
- MORO, C. M.; JR, L. A. M. Cálculo da distância entre diagnósticos cid-10. **Insuficiência cardíaca**, v. 509, p. 313.
- MULLENBACH, J. et al. Explainable prediction of medical codes from clinical text. **arXiv preprint arXiv:1802.05695**, 2018.
- REYS, A. D. et al. Predicting multiple icd-10 codes from brazilian-portuguese clinical notes. In: SPRINGER. **Brazilian Conference on Intelligent Systems**. [S.l.], 2020. p. 566–580.
- SAMMANI, A. et al. Automatic multilabel detection of icd10 codes in dutch cardiology discharge letters using neural networks. **NPJ digital medicine**, Nature Publishing Group, v. 4, n. 1, p. 1–10, 2021.
- SERVICES, C. for M. . M. **2018 ICD-10 CM and GEMs**. 2018. Disponível em: <<https://www.cms.gov/Medicare/Coding/ICD10/2018-ICD-10-CM-and-GEMs>>.

VIRGINIO, L.; REIS, J. C. dos. Automated coding of medical diagnostics from free-text: the role of parameters optimization and imbalanced classes. In: SPRINGER. **International Conference on Data Integration in the Life Sciences**. [S.l.], 2018. p. 122–134.

WANG, S.-m. et al. **Using Deep Learning for Automatic Icd-10 Classification from Free-Text Data**. [S.l.]: International Journal of Medical Research & Health Sciences, 2020.

XU, K. et al. Multimodal machine learning for automated icd coding. In: PMLR. **Machine Learning for Healthcare Conference**. [S.l.], 2019. p. 197–215.

ZHONG, J.; YI, X. Categorizing patient disease into icd-10 with deep learning for semantic text classification. IntechOpen, 2020.